

Finance Billing Analytics System

Capstone Project Technical Documentation Report

Course Tracking: Wipro NGA Training Program (WIPRO_NGA-.Net_FSDAngular-FY27-C1)

Prepared By: Giridhar Gopal

Date of Submission: June 2026

1. Introduction:

Finance Billing Analytics System is a web-based application developed to manage invoices, payments, revenue tracking, audit logging and business analytics. The system provides both MVC and REST API interfaces for managing financial data efficiently.

2. Problem Statement:

Organizations often face difficulties in tracking invoices, monitoring payments and analyzing revenue data. Manual processes increase the chances of errors and revenue leakage.

3. Objectives:

- Manage invoices efficiently
 - Track customer payments
 - Generate revenue analytics
 - Maintain audit logs
 - Provide secure API access
 - Implement modern software engineering practices
-

4. Technology Stack:

Frontend:

- Razor Views
- Bootstrap
- AJAX
- jQuery

Backend:

- ASP.NET Core MVC
- ASP.NET Core Web API

Database:

- SQL Server

Testing:

- xUnit
- Moq

DevOps:

- Docker
- GitHub Actions

5. System Architecture:

Architecture Layers:

Presentation Layer → Controllers & Views

Business Layer → Services

Data Access Layer → Repositories

Database Layer → SQL Server

API Layer → REST APIs with JWT Authentication

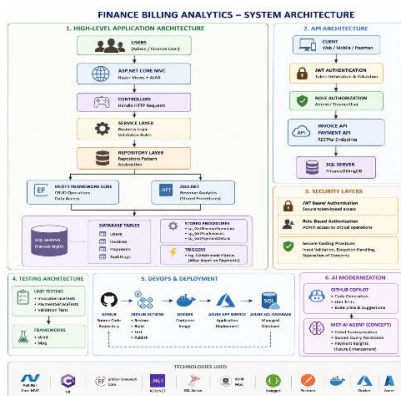


Figure 5 – System Architecture Diagram

5.1 Data Flow Diagram (DFD):

The Data Flow Diagram illustrates how information flows through the Finance Billing Analytics System.

Level 0 DFD:

User interacts with the Finance Billing Analytics System to manage invoices, payments, reports and audit logs. The application processes requests and stores data in SQL Server Database.

Flow:

User → Finance Billing Analytics System → SQL Server Database

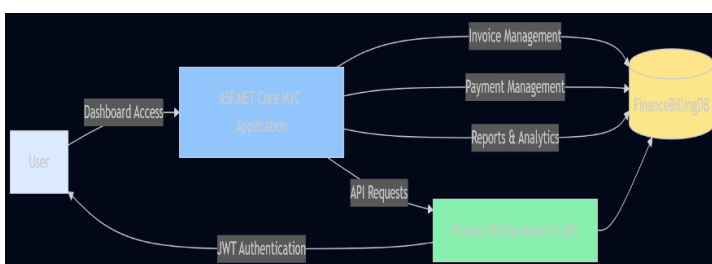


Figure 5.1 – Level-0 Data Flow Diagram (Context Diagram)

Level 1 DFD:

1. User Login Request
2. Authentication Validation
3. Invoice Management Operations
4. Payment Processing Operations
5. Report Generation
6. Audit Logging
7. Database Storage and Retrieval

This layered flow ensures proper separation between presentation, business logic and data access layers.

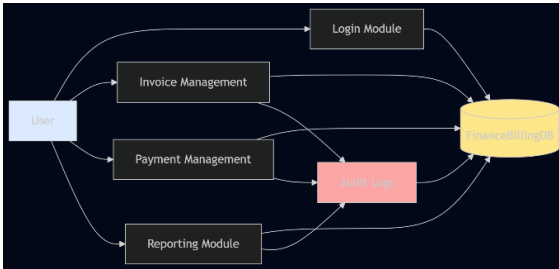


Figure 5.1.1 – Level-1 Data Flow Diagram

6. Database Design:

Tables:

1. Users
2. Invoices
3. Payments
4. AuditLogs

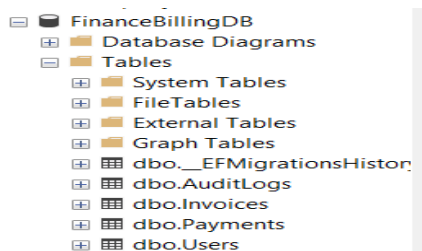


Figure 6 – SQL-Server Database Design

6.1 Entity Relationship (ER) Diagram:

The database design follows relational database principles to maintain data integrity and consistency.

Entities:

Users:

- UserId (Primary Key)
- UserName
- Email
- PasswordHash

Invoices:

- InvoiceId (Primary Key)
- InvoiceNumber
- CustomerName
- InvoiceDate
- Amount
- Status

Payments:

- PaymentId (Primary Key)
- InvoiceId (Foreign Key)
- AmountPaid
- PaymentDate
- PaymentMethod

AuditLogs:

- AuditId (Primary Key)
- Action
- Description
- CreatedDate

Relationships:

- One Invoice can have multiple Payments.
- One Invoice can generate multiple Audit Logs.
- Users authenticate to access system resources.
- Payments automatically update Invoice Status.

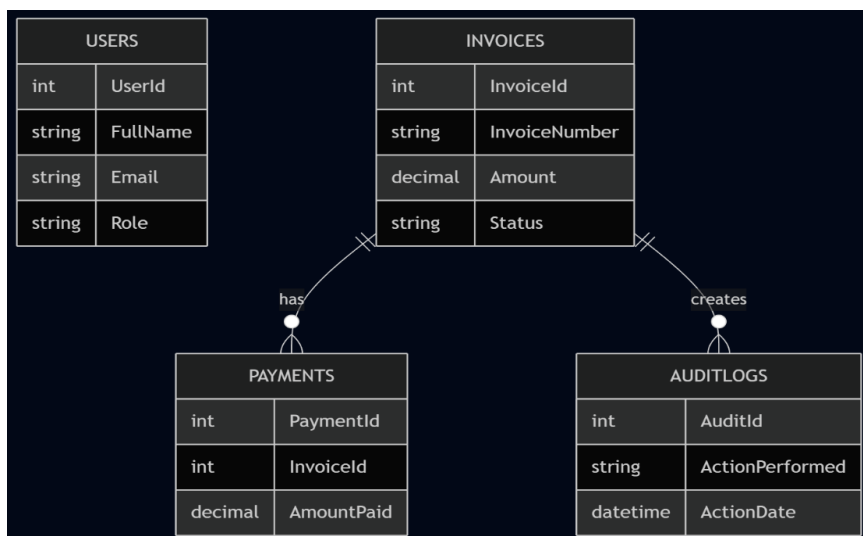


Figure 6.1 - Entity Relationship Diagram showing relationships between Users, Invoices, Payments, and AuditLogs tables in the Finance Billing Analytics System.

7. Project Structure:

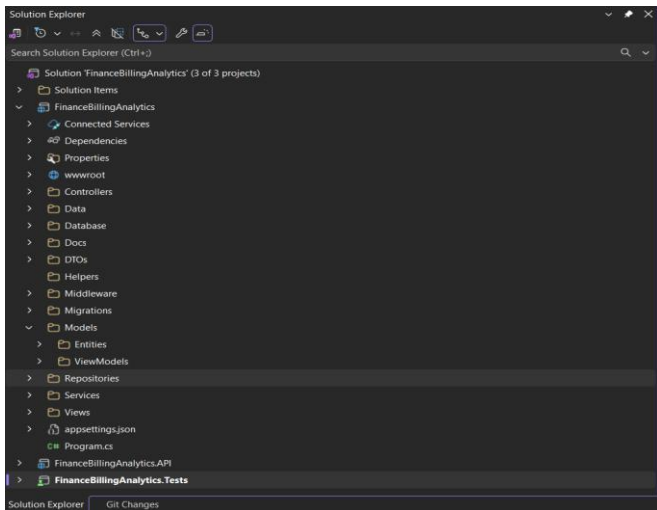


Figure 7.1: Decoupled N-Tier Solution Architecture mapped inside Visual Studio.

8. Features Implemented:

- ✓ Invoice Management
 - ✓ Payment Management
 - ✓ Revenue Dashboard
 - ✓ Audit Logging
 - ✓ Reports
 - ✓ JWT Authentication
 - ✓ AJAX Operations
 - ✓ Unit Testing
-

9. User Stories Implementation:

User Story 1:

Manage Invoices

Implementation:

InvoiceController, InvoiceService, InvoiceRepository

User Story 2:

Manage Payments

Implementation:

PaymentController, PaymentService, PaymentRepository

User Story 3:

Generate Reports

Implementation:

ReportsController, Stored Procedures

User Story 4:

Audit Tracking

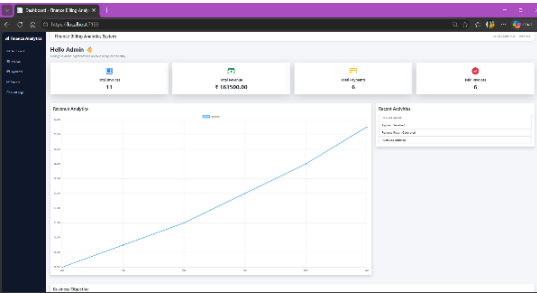
Implementation:

AuditLog Table and SQL Trigger

10. MVC Application Demonstration:

Dashboard Features:

- Total Invoices
- Total Revenue
- Total Payments
- Paid Invoices
- Revenue Analytics
- Recent Activities



Invoice Module:

The screenshot shows the 'Invoice Management' module. It displays a table with columns: 'Invoice Number', 'Customer Name', 'Amount', 'Status', and 'Action'. The table contains 10 rows of invoice data. The 'Status' column uses color-coded indicators: green for 'Paid', yellow for 'Pending', and red for 'Overdue'. The 'Action' column contains links for 'View', 'Edit', and 'Delete'.

Invoice Number	Customer Name	Amount	Status	Action
INV001	ABC Company Ltd	₹ 1,50,000	Paid	View Edit Delete
INV002	XYZ Company	₹ 2,00,000	Pending	View Edit Delete
INV003	DEF Ltd	₹ 1,20,000	Overdue	View Edit Delete
INV004	GHI	₹ 80,000	Paid	View Edit Delete
INV005	JKL	₹ 90,000	Pending	View Edit Delete
INV006	MNO	₹ 1,00,000	Overdue	View Edit Delete
INV007	PQRS	₹ 1,10,000	Paid	View Edit Delete
INV008	RSTU	₹ 1,20,000	Pending	View Edit Delete
INV009	VWXY	₹ 1,30,000	Overdue	View Edit Delete
INV010	ZABC	₹ 1,40,000	Paid	View Edit Delete

Payment Module:

The screenshot shows the 'Payment Management' module. It displays a table with columns: 'Payment ID', 'Invoice ID', 'Amount', 'Payment Date', 'Payment Method', and 'Status'. The table contains 8 rows of payment data. The 'Status' column uses color-coded indicators: green for 'Paid', yellow for 'Pending', and red for 'Overdue'. The 'Payment Method' column lists various methods like 'Bank Transfer', 'Credit Card', and 'Cash'.

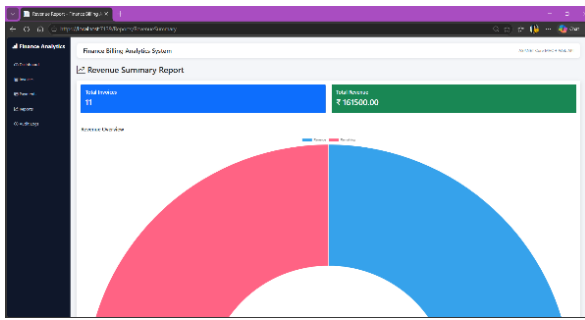
Payment ID	Invoice ID	Amount	Payment Date	Payment Method	Status
PAY001	INV001	₹ 1,50,000	2023-10-25	Bank Transfer	Paid
PAY002	INV002	₹ 2,00,000	2023-10-26	Credit Card	Pending
PAY003	INV003	₹ 1,20,000	2023-10-27	Cash	Overdue
PAY004	INV004	₹ 80,000	2023-10-28	Bank Transfer	Paid
PAY005	INV005	₹ 90,000	2023-10-29	Credit Card	Pending
PAY006	INV006	₹ 1,00,000	2023-10-30	Cash	Overdue
PAY007	INV007	₹ 1,10,000	2023-10-31	Bank Transfer	Paid
PAY008	INV008	₹ 1,20,000	2023-11-01	Credit Card	Pending

Audit Log Module:

The screenshot shows the 'Audit Log' module. It displays a table with columns: 'Action', 'User', and 'Date'. The table contains 8 rows of audit log data. The 'Action' column lists various actions like 'Login', 'Logout', 'Add Invoice', 'Edit Invoice', 'Delete Invoice', 'Add Payment', 'Edit Payment', and 'Delete Payment'. The 'User' column lists the user who performed the action, and the 'Date' column shows the date and time of the action.

Action	User	Date
Login	admin	2023-10-25 10:00:00
Logout	admin	2023-10-25 10:30:00
Add Invoice	admin	2023-10-25 11:00:00
Edit Invoice	admin	2023-10-25 11:30:00
Delete Invoice	admin	2023-10-25 12:00:00
Add Payment	admin	2023-10-25 12:30:00
Edit Payment	admin	2023-10-25 13:00:00
Delete Payment	admin	2023-10-25 13:30:00

Reports Module:



10.1 Front-End and Back-End Connectivity:

The Finance Billing Analytics System follows a layered architecture where the presentation layer communicates with business services and repositories before accessing the database.

MVC Flow:

User Action
↓
Razor View
↓
MVC Controller
↓
Service Layer
↓
Repository Layer
↓
Entity Framework Core
↓
SQL Server Database

API Flow:

Postman / Swagger
↓
API Controller
↓
Service Layer
↓
Repository Layer
↓
SQL Server Database

This architecture ensures maintainability, scalability and separation of concerns while supporting both MVC and Web API interfaces.

11. Web API Demonstration:

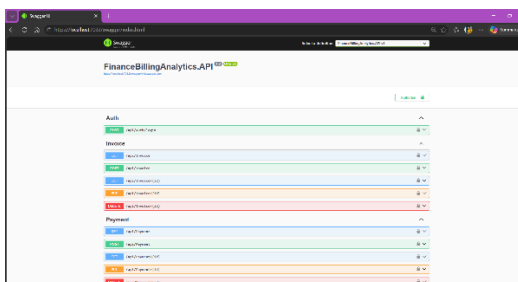


Figure 11 - Swagger API

11.1 API Documentation Summary:

The project exposes RESTful Web APIs secured using JWT Authentication.

Method	Endpoint	Description
POST	api/Auth/login	User Authentication
GET	api/Invoice	Retrieve all invoices
GET	api/Invoice/{id}	Retrieve invoice by ID
POST	api/Invoice	Create invoice
PUT	api/Invoice/{id}	Update invoice
DELETE	api/Invoice/{id}	Delete invoice
GET	api/Payment	Retrieve payments
POST	api/Payment	Create payment

12. JWT Authentication and Authorization:

Security is implemented using JSON Web Tokens (JWT) to protect API endpoints and ensure authenticated access.

Authentication Workflow:

1. User enters login credentials.
2. Credentials are validated against database records.
3. JWT Token is generated upon successful authentication.
4. Token is returned to the client.
5. Client sends token with every protected API request.
6. Token validation middleware verifies authenticity.
7. Authorized access is granted.

Components Used:

- AuthController
- JwtService
- JWT Middleware
- Authorization Attributes
- Swagger Authorization Support

Benefits:

- Stateless Authentication
- Secure API Access
- Improved Scalability
- Industry Standard Security Implementation

The implementation follows modern enterprise authentication practices commonly used in production systems.

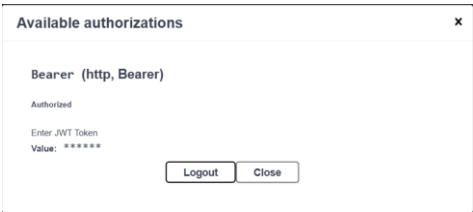


Figure 12.1 - Swagger UI dashboard demonstrating documented endpoints and the JWT Bearer authorization configuration.

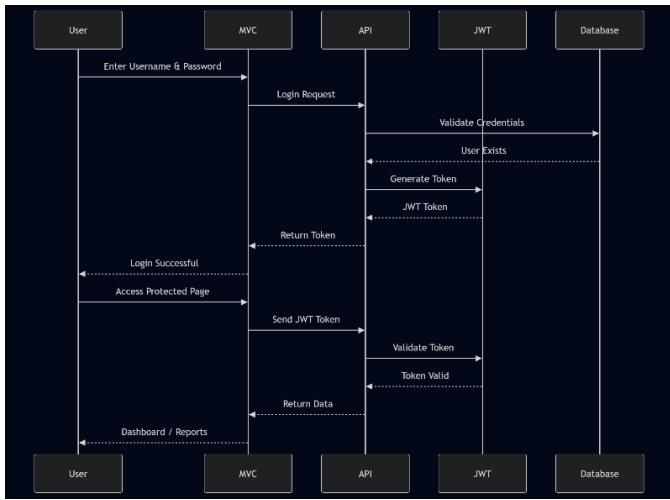


Figure 12.2 - JWT Workflow

13. AJAX Implementation:

AJAX is used to perform asynchronous operations without full page reload.

Implementation Location:

- Invoice Index View
- Payment Index View

Benefits:

- Faster User Experience
- Reduced Page Reloads

13.1 Report Generation Process:

The reporting module provides business insights using data collected from invoices and payments.

Process Flow:

1. User accesses Reports Module.
2. Reports Controller receives request.
3. Service Layer processes business logic.
4. Stored Procedure executes revenue calculations.
5. SQL Server returns aggregated data.
6. Dashboard displays charts using Chart.js.
7. Users analyze business performance metrics.

Reports Generated:

- Revenue Summary
- Payment Analytics
- Invoice Status Distribution
- Business Performance Metrics

The report generation process helps management make data-driven business decisions.

14. Repository Pattern Implementation:

Repositories Implemented:

- IInvoiceRepository
- InvoiceRepository
- IPaymentRepository
- PaymentRepository
- IUserRepository

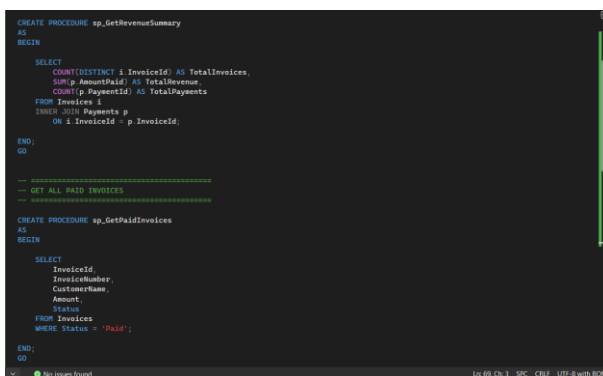
Benefits:

- Separation of Concerns
- Better Maintainability
- Easier Unit Testing

15. Stored Procedures:

Purpose:

Generate revenue analytics.



```
CREATE PROCEDURE sp_GetRevenueSummary
AS
BEGIN
    SELECT
        COUNT(DISTINCT i.InvoiceId) AS TotalInvoices,
        SUM(p.AmountPaid) AS TotalRevenue,
        COUNT(p.PaymentId) AS TotalPayments
    FROM Invoices i
    INNER JOIN Payments p
    ON i.InvoiceId = p.InvoiceId
END
GO

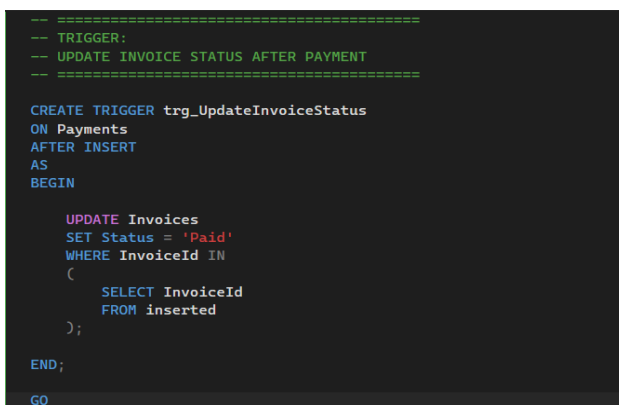
-- GET ALL PAID INVOICES
-- =====
CREATE PROCEDURE sp_GetPaidInvoices
AS
BEGIN
    SELECT
        InvoiceId,
        InvoiceNumber,
        CustomerName,
        Amount,
        Status
    FROM Invoices
    WHERE Status = 'Paid'
END
GO
```

Figure 15 - Stored Procedure

16. Triggers:

Trigger Purpose:

Automatically create audit records when invoice/payment data changes.



```
-- =====
-- TRIGGER:
-- UPDATE INVOICE STATUS AFTER PAYMENT
-- =====

CREATE TRIGGER trg_UpdateInvoiceStatus
ON Payments
AFTER INSERT
AS
BEGIN
    UPDATE Invoices
    SET Status = 'Paid'
    WHERE InvoiceId IN
    (
        SELECT InvoiceId
        FROM inserted
    );
END
GO
```

Figure 16 - Triggers

16.1 Business Rules Summary:

The Finance Billing Analytics System enforces the following business rules:

Invoice Rules:

- Invoice Number must be unique.
- Invoice Amount must be greater than zero.
- Invoice Status can be Pending or Paid.

Payment Rules:

- Payment must be linked to an existing invoice.
- Payment Amount cannot exceed Invoice Amount.
- Full payment automatically updates invoice status to Paid.

Audit Rules:

- All critical transactions are recorded.
- Invoice and payment modifications generate audit logs.

Authentication Rules:

- Protected APIs require valid JWT tokens.
- Unauthorized users cannot access secured resources.

These rules ensure data consistency, reliability and business compliance.

17. Middleware:

Custom Exception Middleware is implemented to handle application exceptions centrally.

File:

ExceptionMiddleware.cs

Benefits:

- Centralized Error Handling
 - Better Maintainability
-

18. Unit Testing:

Frameworks Used:

- xUnit
- Moq

Implemented Tests:

- InvoiceServiceTests
- PaymentServiceTests
- ModelValidationTests

Result:

12 Tests Passed

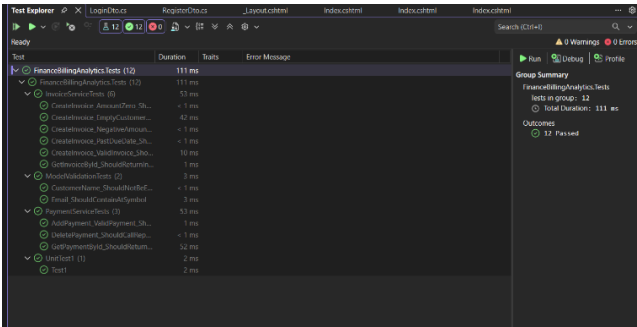


Figure 18 - Visual Studio Test Explorer demonstrating successful execution of the isolated xUnit automated testing suite

19. Docker Implementation:

Files:

- Dockerfile
- .dockerignore

Benefits:

- Consistent Deployment
- Containerized Environment

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

WORKDIR /src

COPY . .

RUN dotnet restore

RUN dotnet publish FinanceBillingAnalytics/FinanceBillingAnalytics.csproj -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0

WORKDIR /app

COPY --from=build /app/publish .

ENTRYPOINT ["dotnet", "FinanceBillingAnalytics.dll"]
```

20. GitHub Actions CI/CD:

Workflow File:

.github/workflows/dotnet.yml

Purpose:

Automatic Build and Test Execution

```
name: .NET Build

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      - name: Restore
        run: dotnet restore

      - name: Build
        run: dotnet build --no-restore

      - name: Test
        run: dotnet test --no-build
```

20.1 GitHub Repository Health:

The source code is maintained with clean repository health standards, including logical folder decoupling, explicit .gitignore definitions (excluding bin, obj, and local database files), and structured commit histories.

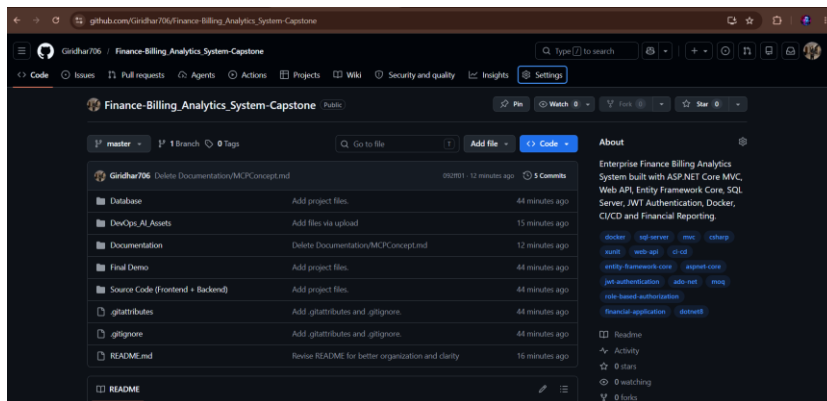


Figure 20.1 - Version control repository showcasing clean file structures and cloud backup readiness

21. Industry Best Practices Followed:

The project was developed following industry-standard software engineering principles.

Architecture Practices:

- Layered Architecture
- Repository Pattern
- Dependency Injection
- Service-Oriented Design

Security Practices:

- JWT Authentication
- API Authorization
- Secure Configuration Management
- HTTPS Support

Quality Practices:

- Unit Testing
- Centralized Exception Handling
- Input Validation
- Code Reusability

DevOps Practices:

- Docker Containerization
- Git Version Control
- GitHub Actions CI/CD
- Automated Build Verification

These practices improve maintainability, scalability and overall software quality.

22. Challenges Faced:

- JWT Authentication and Authorization Configuration
 - Implementing Secure Token-Based API Access
 - Repository Pattern Integration and Dependency Injection
 - Entity Framework Core Migrations and Database Synchronization
 - Designing a Scalable Layered Architecture
 - Establishing MVC and Web API Communication
 - Managing SQL Server Stored Procedures and Queries
 - AJAX-Based Dynamic Data Loading and UI Updates
 - Error Handling and Exception Management
 - Dashboard Data Visualization using Charts
 - Unit Testing Setup using xUnit and Moq
 - API Testing and Validation using Postman and Swagger
 - Maintaining Audit Logs and Data Traceability
 - Optimizing Database Performance and Query Execution
 - Implementing Role-Based Access Control (RBAC)
 - Generating Revenue Reports and Analytics Dashboards
 - Debugging Integration Issues Across Multiple Layers
-

23. Assumptions and Constraints:

The following assumptions were considered during development:

- Users possess valid credentials.
- SQL Server Database is continuously available.
- Internet connectivity is stable.
- Revenue calculations are based on invoice and payment data.
- Application is intended for small and medium-sized organizations.

These assumptions helped simplify development while focusing on core business requirements.

24. Learning Outcomes:

- ASP.NET Core MVC Development
 - REST API Development
 - JWT Authentication
 - Entity Framework Core
 - SQL Server Programming
 - Unit Testing
 - Docker Basics
 - CI/CD Fundamentals
-

25.Comparison with Real-World Applications:

The Finance Billing Analytics System implements core functionalities commonly found in enterprise financial software.

Feature	Finance Billing Analytics	SAP Finance	Zoho Books
Invoice Management	Yes	Yes	Yes
Payment Tracking	Yes	Yes	Yes
Revenue Analytics	Yes	Yes	Yes
Audit Logs	Yes	Yes	Yes
JWT Security	Yes	Yes	Yes
CI/CD Integration	Yes	Yes	Yes
Cloud Deployment	Future Scope	Yes	Yes
Payment Gateway	Future Scope	Yes	Yes
AI Analytics	Future Scope	Partial	Partial

The project successfully demonstrates the core architecture and workflow used in modern financial management systems while remaining lightweight and educational in nature.

26.Future Scope:

The current Finance Billing Analytics System successfully manages invoices, payments, reporting and secure APIs. In future, the system can be enhanced with the following features:

1. Cloud Deployment:

- Deploy the application on Microsoft Azure App Services.
- Use Azure SQL Database for managed database services.
- Configure automated cloud deployments through GitHub Actions.

2. Advanced Analytics Dashboard:

- Generate monthly, quarterly and yearly revenue trends.
- Provide predictive analytics for revenue forecasting.

3. Role-Based User Management:

- Introduce multiple user roles such as Admin, Accountant and Auditor.
- Implement role-specific dashboards and permissions.

4. Email & Notification System:

- Send automated invoice reminders.
- Notify customers about payment due dates.
- Generate email alerts for overdue invoices.

5. Payment Gateway Integration:

- Integrate Razorpay, Stripe or PayPal.
- Enable real-time online payment processing.
- Automatically update invoice status after successful payments.

6. Enhanced Security:

- Multi-Factor Authentication (MFA).
 - OAuth 2.0 and Single Sign-On (SSO).
 - Advanced audit monitoring and security reporting.
-

27. Conclusion:

The **Finance Billing Analytics System** was successfully designed and developed as a comprehensive financial management solution that streamlines invoice processing, payment tracking, revenue monitoring, audit logging, and business reporting. The application provides a centralized platform for managing critical financial operations while ensuring accuracy, transparency, and security across all transactions.

The project demonstrates the practical implementation of modern software engineering principles through a well-structured **layered architecture, Repository Pattern, Entity Framework Core, and ASP.NET Core MVC & Web API** integration. Security has been strengthened using **JWT-based Authentication and Role-Based Authorization**, ensuring that only authorized users can access protected resources and perform sensitive operations.

From a database perspective, the system effectively utilizes **SQL Server**, relational database design, stored procedures, joins, triggers, and optimized data access mechanisms to maintain data integrity and support efficient reporting. The inclusion of audit logging enables traceability of user actions, which is an important requirement for enterprise-grade financial systems.

The project also incorporates industry-standard development practices such as **Unit Testing with xUnit and Moq, Docker Containerization, GitHub Actions CI/CD Automation**, and comprehensive API testing using **Swagger UI and Postman**. These practices improve application reliability, maintainability, deployment consistency, and overall software quality.

28. Developer's Retrospective & Declaration:

28.1 Declaration of Originality:

I, **Giridhar Gopal**, hereby declare that this capstone project, titled "**Finance Billing Analytics System**" is my original work developed as part of the Wipro NGA Training Program. The system architecture, database design, APIs, business logic, user interface, and documentation presented in this report were designed and implemented by me to fulfill the project requirements. Industry-standard technologies and frameworks were used wherever appropriate, and all external resources have been duly acknowledged.

28.2 Developer's Learning Journey:

Developing the Finance Billing Analytics System was a highly rewarding learning experience that enhanced my understanding of enterprise application development. Through this project, I gained hands-on experience with **ASP.NET Core MVC, Web APIs, SQL Server, Entity Framework Core, JWT Authentication, Docker, and CI/CD practices**.

Implementing layered architecture, secure authentication, RESTful APIs, reporting modules, and audit logging helped me understand how modern financial applications are designed and maintained. This project strengthened my technical skills, problem-solving abilities, and confidence in building scalable, secure, and industry-ready software solutions.

29. Acknowledgements:

I would like to express my sincere gratitude to my mentor, **Mr. Parth Shukla**, for his valuable guidance, technical expertise, and continuous support throughout the development of this project. His feedback and encouragement played a significant role in helping me successfully complete this capstone project.

I would also like to thank the **Wipro NGA Training Team** and evaluators for providing the learning resources, practical exposure, and opportunities that enabled me to apply industry-standard technologies and best practices while building this application.

Name: Giridhar Gopal

Date: 05 June 2026